FastGraph: PCA-Subspace Binned k-Nearest-Neighbor Graph Construction for GPU Geometric Deep Learning

Aarush Agarwal[a,*], Jan Kieseler[b], Raymond He[a], Matteo Cremonesi[a], Shah Rukh Qasim[c]

[a] *Carnegie Mellon University, Pittsburgh, PA, USA*
[b] *Karlsruhe Institute of Technology, Karlsruhe, Germany*
[c] *University of Zurich, Zürich, Switzerland*

Abstract

Dynamic graph neural networks repeatedly construct k-nearest-neighbor graphs in learned latent spaces, making GPU-accelerated graph construction an important operation in scientific point-cloud workloads. We present **FastGraph**, a PyTorch-integrated primitive for exact kNN graph construction in low-to-moderate-dimensional spaces. FastGraph uses a PCA-subspace cell list: it fits a compact binning coordinate system to the input tensor, uses that subspace for spatial pruning, and evaluates all candidate distances in the original feature space. The data-adaptive orthonormal subspace improves uniform-grid pruning in the d=4–10 range evaluated around the d=4 GravNet operating point. Point data remain GPU-resident, with small host synchronizations for control metadata. The implementation ships with an eager-mode PCA API and an opt-in differentiable `GravNetOp` integration. On a 5 M-point, single-segment detector-derived HGCAL recHit stress test, FastGraph builds the exact graph in 7.5 s at d=8, k=40, achieving speedups of up to 41× over FAISS-GPU, 19× over cuVS brute force, and 17× over approximate CAGRA while retaining exact neighbor sets under the paper's distance-based convention.

Keywords: Graph Neural Networks, k-Nearest Neighbors, GPU Acceleration, CUDA, PCA, Particle Physics

1. Introduction

Graph neural networks (GNNs) have become a standard substrate for reasoning over irregular relational data, from web-scale recommender systems [1] to particle reconstruction in high-granularity calorimeters [2, 3]. A defining architectural choice in many modern GNNs is that the graph itself is built at training time, from a learned latent embedding, by a k-nearest-neighbor query. The dynamic kNN graph is constructed on every forward pass, and its cost can be a substantial component of a model step. Establishing that fraction for the target HGCAL models requires end-to-end profiling and is outside the kernel study reported here. Importantly, the latent dimensionality in the target application is low-to-moderate-dimensional: the original GravNet layer for HGCAL reconstruction [2] uses d=4 for the learned space in which the kNN is taken. Related multi-particle and object-condensation models used in CMS HGCAL reconstruction [3, 4, 5] motivate evaluating the nearby d=4–10 range. EdgeConv [6] and ParticleNet [7] also rebuild feature-space graphs, but later layers may operate at substantially higher dimensionality and are outside the primary regime evaluated here. This *low-to-moderate-dimensional* regime is precisely where neither generic high-dimensional ANN libraries nor brute-force GPU kNN are an ideal fit: ANN libraries introduce unfavorable overheads, and brute force scales as N^2d regardless of how much spatial structure the latent embedding contains.

Why exact kNN matters in this regime. Exact kNN provides a fixed distance criterion without an index-specific recall target or quality hyperparameter. Tied distances may still admit multiple valid exact neighbor sets unless a tie-breaking rule is fixed. Approximate construction changes the selected edge set, and graph-learning studies show

*Corresponding author

Email addresses: `aarusha@andrew.cmu.edu` (Aarush Agarwal), `jan.kieseler@cern.ch` (Jan Kieseler), `rhe2@andrew.cmu.edu` (Raymond He), `mcremone@andrew.cmu.edu` (Matteo Cremonesi), `shah.rukh.qasim@cern.ch` (Shah Rukh Qasim)

that edge perturbations can alter downstream predictions substantially [8, 9]. The present work does not establish that approximate-kNN errors produce a measurable HGCAL physics systematic; that question requires an end-to-end model study. We instead evaluate the narrower engineering case for exact construction when it is competitive in the target low-to-moderate-dimensional regime.

The classical low-dimensional accelerator for this regime is the *cell list* [10, 11]: place points in a uniform grid, expand neighboring bins around each query until the best-k radius is certified, and evaluate distances only for candidates in the visited bins. A full d-dimensional grid scales as n_{bins}^d, so practical GPU implementations limit the binning grid to a small k even when the input has $d > k$ coordinates. FastGraph addresses this constraint with a data-adaptive choice of binning coordinates: it bins in the top-k principal subspace while retaining full-dimensional distances for candidate evaluation. Since the partial projection $V_k V_k^\top \preceq I_d$ is contractive, projected-space termination remains a valid lower bound on true-space distance, so the algorithm preserves exactness for supported input dimensions. The PCA formulation is evaluated at d=4–10; d=2–3 are axis-aligned controls for which the implementation short-circuits before PCA. At d=8, N=5 M, k=40, FastGraph builds the kNN graph in 7.5 s versus 310 s for exact FAISS-GPU, 146 s for cuVS brute force, and 127 s for CAGRA's approximate NN-Descent build. Across d=2–10 at this (N, k) point, geomean speedups are 10.4×, 5.0×, and 4.4× respectively.

The library is shipped as a PyTorch extension. The axis-aligned primitive remains callable from TorchScript, while the PCA wrapper uses eager-mode `torch.pca_lowrank`. Gradients flow through the continuous outputs of the kernel (selected squared distances, query coordinates) while the hard neighbor indices are treated as constants in the backward pass. `GravNetOp` [2] exposes PCA-subspace neighbor selection through `use_pca=True` in eager mode.

2. Related Work

Exact GPU kNN. Modern GPUs make the inner-loop distance computation of exact kNN embarrassingly parallel. FAISS [12] provides a highly tuned `IndexFlatL2` exact brute-force GPU index together with the fast k-selection primitives that downstream methods reuse. RAPIDS `cuVS` exposes a comparable exact brute-force GPU implementation that we use as the strongest exact baseline in this work; KeOps [13] provides symbolic matrix reductions with automatic differentiation that can express exact kNN without materializing the full distance matrix. Direct exact GPU kNN-graph construction has also been studied in the multi-GPU and cluster settings [14, 15]. PyTorch Geometric exposes the widely used `knn_graph` interface through its `torch_cluster` extension [16]; we treat it as a framework interface rather than a distinct algorithmic baseline. FastGraph is not the first exact GPU kNN library; it specializes the accelerator for the small-d exact graph-construction setting that arises in dynamic GNN layers.

Cell lists and spatial binning. The cell list is the canonical low-dimensional accelerator: it dates to Verlet's molecular-dynamics neighbor-finding scheme and underpins modern MD libraries such as HOOMD-blue. The construction is the same across communities—uniform grid in d dimensions, sort points by bin, expand a concentric hypercube of bins around each query until the best-k radius is certified—but the choice of d has been constrained in practice to two or three axis-aligned dimensions because the bin tensor scales as n_{bins}^d. Qasim's PhD thesis [17] discusses the practical grid dimensionality limit in the GNN context. FastGraph uses a principal subspace for the grid coordinates; an axis-aligned first-column choice is retained only as an internal ablation.

PCA-projected indexing. Using a principal-component projection as a coordinate system for spatial indexing has a long history; Sproull's k-d tree refinements [18] already noted that splitting on the direction of maximum variance yields more balanced partitions than splitting on a coordinate axis. Principal-axis tree variants use a similar idea for CPU-side neighbor search. FastGraph's PCA-subspace binning differs in two ways: the projection is partial (top-k axes) rather than full, and the candidate-distance evaluation that follows the pruning step is performed in the *original* d-dimensional space, preserving exactness. The contraction $V_k V_k^\top \preceq I_d$ is what makes that combination valid (Section 3).

Three-dimensional spatial-acceleration kNN. A separate line of work targets exact GPU kNN through dedicated spatial-acceleration structures that exploit the GPU's ray-tracing hardware or BVH machinery. RTNN [19], RT-kNNS Unbound [20], and Arkade [21] retarget the OptiX BVH unit for neighbor search and report large speedups over shader-core baselines in 3D. LBVH-kNN [22] (`cupy-knn`) attains similar gains via a left-balanced LBVH built without dedicated RT hardware. CLOVER [23] introduces a GPU-native, Voronoi spatio-graph approach to exact kNN and reports 4× over an "optimized grid" baseline and 230× over FAISS on low-dimensional spatial data. All of these methods are implemented and evaluated for two- or three-dimensional inputs: Arkade notes that RT cores build BVHs only on three-dimensional data, CLOVER's reference implementation asserts `D == 3`, and LBVH-kNN is evaluated exclusively in 3D. We include a direct head-to-head against CLOVER at d=3 in Section 5.6, but the released methods do not provide a d=4–10 path comparable to the one evaluated here.

Approximate GPU kNN. A parallel body of work has produced GPU-resident approximate nearest neighbor (ANN) systems. SONG [24] decomposes graph traversal into GPU-friendly stages; GGNN [25] builds a hierarchical kNN graph for index construction and search; CAGRA [26], distributed as part of RAPIDS `cuVS`, builds a high-quality graph index with an IVF-PQ or NN-Descent initialization. ANN-Descent variants have also been adapted to reduce memory traffic during construction [27]. These systems expose a tunable recall–throughput tradeoff for high-throughput vector retrieval; FastGraph instead targets exact, small-d graph construction. We benchmark CAGRA and GGNN as approximate comparators (Section 5.7) but do not include CPU-only ANN systems (HNSW, ScaNN, Annoy) [28, 29, 30] in the timing tables: the point tensors and large intermediate arrays remain GPU-resident, and a CPU-side detour for bulk data would not be a fair comparator.

Dynamic latent-graph models in geometric deep learning. Dynamic kNN-graph layers in learned latent spaces are a recurring ingredient in geometric deep learning. DGCNN/EdgeConv recomputes the graph in feature space at each layer [6]; ParticleNet adapts the same idea to jet tagging [7]; GravNet/GarNet layers [2] and the object condensation loss [3] are deployed for HGCAL reconstruction [4, 5]; the HEP.TrkX/Exa.TrkX line of work uses learned-embedding graphs for tracking [31, 32, 33]. FastGraph is best framed as an exact substrate for the kNN-graph layer inside such models—not a new model class.

3. The PCA-Subspace Binned kNN Algorithm

3.1. Phase 1: Subspace estimation

Given the input coordinates $X \in \mathbb{R}^{N\times d}$ for the current call, FastGraph samples up to 50,000 rows, centers that sample, and computes the top-k principal axes with `torch.pca_lowrank`. The current release fits one projection for the complete coordinate tensor rather than a separate projection for each `row_splits` segment. The number of binning axes k is a user-facing hyperparameter; we default to $k = 3$ for the HGCAL workloads in this paper, justified empirically in the ablation of Section 5.5. The resulting projection matrix $V_k \in \mathbb{R}^{d\times k}$ satisfies $V_k^\top V_k = I_k$ and $V_k V_k^\top \preceq I_d$. The projected coordinates used for binning are $Y = XV_k$. PCA estimation is performed once per call on the GPU and is included in the reported FastGraph timing. The random subsample follows PyTorch's active random number-generator state.

Short-circuit at $d \le k$. When the input dimensionality d does not exceed the binning dimensionality k, FastGraph skips PCA estimation entirely and binds the binning axes to the raw input axes; the projection V_k reduces to the identity (up to permutation), $Y = X$, and the algorithm behaves exactly as a classical axis-aligned cell list. With the default $k = 3$, the $d \in \{2, 3\}$ gains therefore come from the underlying cell-list machinery; the PCA contribution begins at $d > k$, which covers the d=4–10 HGCAL latent-space range.

3.2. Phase 2: Binning in the projected subspace

We partition Y with a uniform grid whose scalar width w is shared across the k projected axes. The division count is set heuristically to balance bin density against neighborhood reach,

$$n_{\text{bins}} = \left(\frac{32\, n_{\text{elems}}}{K}\right)^{\frac{1}{k}},$$

where the released implementation uses the historical occupancy proxy $n_{\text{elems}} = \lfloor \texttt{max(row\_splits)}/|\texttt{row\_splits}| \rfloor + 1$, K is the requested neighborhood size, and the integer result is clamped to $[5, 30]$. For the one-segment benchmark input `row_splits`$=[0, N]$, this proxy is approximately $N/2$; it is a performance heuristic and does not enter the exactness condition. Points are sorted by bin index, and cumulative offsets are stored so that each bin can be addressed as a contiguous slice of the sorted point array. Row splits, which delineate the per-event boundaries inside a batched tensor, are honored by the binning kernel so that no query bin spans events.

3.3. *Phase 3: Per-query search with full-dim distances*

For each query x_q, the search starts in the bin containing $y_q = x_q V_k$ and expands through successive shells of k-D hypercube bins. For every candidate u in a visited bin the *full-dimensional* squared Euclidean distance $\|x_q - x_u\|^2 = \sum_{i=1}^{d}(x_{q,i} - x_{u,i})^2$ is computed (not the k-dimensional projected distance). The current best neighbor radius r_K and a fixed-size top-K candidate buffer are maintained incrementally.

3.4. *Why this is exact: the contraction argument*

For finite inputs with valid row splits, a supported projection dimension $2 \le k \le d$, and at least K eligible points in each segment (or in its direction-masked subset), consider the standard cube-radius termination test applied to bins in the projected subspace while all candidate distances are evaluated in the full input space. The exactness of this test rests on a single observation about the partial projection. For any two points $x_a, x_b \in \mathbb{R}^d$,

$$\|y_a - y_b\|^2 = (x_a - x_b)^\top V_k V_k^\top (x_a - x_b) \le \|x_a - x_b\|^2,$$

because $V_k V_k^\top$ has eigenvalues in $\{0, 1\}$ and is therefore $\preceq I_d$. Equivalently, the projection is contractive: the projected distance never overestimates the true distance.

After visiting all bins within the current hypercube shell of radius s, any unvisited bin lies at projected distance at least $w \cdot s$ from y_q, where w is the scalar width shared by all projected axes in the released implementation. Thus any unvisited point satisfies $\|y_q - y_u\|^2 \ge (w \cdot s)^2$. Combined with the contraction above, $\|x_q - x_u\|^2 \ge (w \cdot s)^2$, so once the top-$K$ buffer has filled and $(w \cdot s)^2 > r_K^2$, no unseen point can be closer in the true distance than the current K-th neighbor. The algorithm may safely terminate; the returned K neighbors are exact under the paper's distance convention. We additionally verify exactness empirically against brute force at tractable scale using the tie-aware distance convention of Section 5.7, obtaining recall = 1.0 with distances matching the FP32 reference within the stated numerical tolerances across all spot-checked configurations.

3.5. *Binstepper*

The `binstepper` is the small bookkeeping utility that enumerates the surface of the s-th hypercube shell around the query bin. It linearises each shell into a flat sequence, converts each step back into per-dimension offsets, discards interior cells, and yields the next surface bin's flat index. The stepper carries no algorithmic role; the kNN guarantee lives in the outer kernel of Algorithm 1. Its job is purely to enumerate the projected shells in a deterministic flat order so the distance kernel can stay focused on the full-dimensional checks. Keeping the shell enumerator separate from the distance kernel also makes the projected walk event-local: the same row-split offsets and contiguous bin slices can be reused across all queries in the batch.

3.6. *Pseudocode*

Algorithm 1 makes the projected-vs-original-space distinction explicit: the bin indices (`binIdx`) and the stepper operate on y-space; the distance function `calcDist` operates on the original x-space coordinates; the termination test references the projected-space cube radius; the contraction argument certifies the result. This separation keeps the implementation and the proof aligned: projected-space search only decides which bins to visit, while exactness is enforced by the original-space distance computation.

3.7. *Complexity*

Let $B = n_{\text{bins}}^k$ be the size of the bin grid. The PCA projection reduces this from the n_{bins}^d of an axis-aligned scheme to a quantity that depends only on the binning subspace size k, *independently of the input dimension d*. This bounds the grid size without tying pruning to the first k input columns.

Setup. Let $S = \min(N, 50{,}000)$ be the PCA sample size and $q = \min(k + 2, d)$ the low-rank solve size. For the fixed number of randomized iterations used by `torch.pca_lowrank`, the subspace fit costs $O(Sdq + (S + d)q^2)$, followed by $O(Ndk)$ to project all points. This is a sampled low-rank computation, not a full $O(Nd^2 + d^3)$ eigendecomposition. Binning, sorting, and offset construction take $O(N \log N + B)$.

Memory. Auxiliary memory is $O(N(d + k + K) + B)$: sorted coordinates, projected coordinates, permutation, neighbor indices, and bin offsets.

Algorithm 1: PCA-Subspace Binned-Select-kNN: bins live in projected y-space; distances are computed in original x-space. Slot 0 is the query point itself, matching the FastGraph tensor convention; downstream message-passing code may drop or ignore this self edge.

```
Input: coords[n_v][n_c] (original-space X), V_k, binIdx[n_v] (projected-space bins), binOffsets[n_v][n_b+1],
       binBounds[n_B], binCounts[n_b], scalar binWidth, n_v, K, n_c, n_b, n_B
Output: neighbors[n_v][K] indices including slot 0 self by convention, r_K^2 values
for each v ∈ [0, n_v) in parallel do
    neighbors[v][0] ← (v, 0); filled ← 1; (maxIdx, maxD2) ← (0, 0)
    totalBins ← ∏_{d=0}^{n_b−1} binCounts[d], gOff ← totalBins · ⌊binIdx[v]/totalBins⌋
    step ← BinStepper(binCounts, binOffsets[v][1..n_b])      // operates in projected y-space
    w ← binWidth
    cont ← true; s ← 0                    // s: current cube-shell radius (projected space)
    while cont do
        step.setDistance(s); cont ← false
        while (off ← step.step()) ≠ −1 do
            idx ← off + gOff
            if idx < 0 or idx ≥ n_B − 1 then
                continue
            (s_b, e_b) ← (binBounds[idx], binBounds[idx+1])
            if s_b = e_b then
                cont ← true; continue
            for u ∈ [s_b, e_b) do
                if u=v then
                    continue
                d2 ← calcDist(v, u)               // full d-dim Σ_i (x_{v,i} − x_{u,i})^2, NOT projected
                if filled < K then
                    neighbors[v][filled] ← (u, d2); if d2 > maxD2 then
                        (maxIdx, maxD2) ← (filled, d2)
                    ; filled++
                else if d2 < maxD2 then
                    neighbors[v][maxIdx] ← (u, d2); (maxIdx, maxD2) ← findMaxDist(neighbors[v])
            cont ← true
        if filled=K and (w · s)^2 > maxD2 then
            break
        s++
```

Per-query search. Under roughly uniform projected-space density, each query examines $O(K)$ useful candidates before certifying r_K; each candidate costs $O(d)$ for the full-dim distance and at most $O(K)$ for the top-K buffer update, giving expected $O(K(d+K))$ per query. The worst case is the degenerate all-points-in-one-bin configuration, which falls back to the brute-force $O(N(d+K))$. In the dynamic-graph setting, the setup terms are amortized over the full query sweep on each event, so the per-query behavior is the more important operational quantity.

Parallel cost. With P CUDA threads,

$$\frac{N}{P} \cdot O(K(d+K)) \; + \; O\Big(S\,dq + (S+d)q^2 + Ndk + N\log N + B\Big).$$

3.8. *Gradient flow and PyTorch integration*

FastGraph exposes gradients through the continuous outputs of the kernel—the selected squared distances and the input coordinates— while treating the discrete neighbor indices and the projection matrix V_k as constants in the backward pass. Concretely, the forward returns $(\mathrm{idx}, \mathrm{dist}^2)$ and the backward applies the chain rule with respect to dist^2 and the input coordinates, yielding a subgradient of the piecewise-smooth loss induced by the hard neighbor selection. This is the same gradient model used by any fixed-index distance layer (e.g., `torch.cdist` followed by `topk`) and integrates naturally into existing GNN autograd graphs. Treating V_k as fixed per forward–backward pair keeps graph selection consistent within that pair; the projection is refreshed at the next forward. The CUDA backward uses atomic accumulation and is therefore not guaranteed to be bitwise deterministic.

The PCA entry point is an eager-mode Python wrapper around the scripted axis-aligned primitive and the custom CUDA operation. A minimal usage example is:

```python
from fastgraphcompute import binned_select_knn_pca
from fastgraphcompute.gnn_ops import GravNetOp

# Direct call (eager mode)
idx, dist2 = binned_select_knn_pca(K=40, coords=x, row_splits=rs,
                                   max_bin_dims=3)
# idx:   (N, K) int64 neighbor indices (constant in backward)
# dist2: (N, K) float32 squared distances (differentiable)
loss = model(dist2).sum()
loss.backward()  # gradients flow through dist2 to x

# Opt-in PCA neighbor construction inside GravNetOp (eager mode)
layer = GravNetOp(in_channels=16, out_channels=32,
                  space_dimensions=8, propagate_dimensions=32,
                  k=40, use_pca=True, max_bin_dims=3)
out, idx, dist2, space = layer(features, rs)
```

Listing 1: Minimal eager-mode PCA-FastGraph usage.

4. Experimental Setup

All GPU experiments use a single NVIDIA A100 80 GB PCIe with CUDA 12.1 and PyTorch 2.5. Compared backends are exact FAISS-GPU `IndexFlatL2` [12], cuVS brute force (also exact), CAGRA via cuVS [26] (approximate), and GGNN [25] (approximate); FastGraph is compiled from the manuscript's `paper-release` branch.

Timing protocol. All comparative HGCAL timing plots and the headline table use the completed matched-input, serialized `-gres=mps:100` campaign only; older `mps:50` and exclusive-allocation rows are excluded. Every backend has seven repetitions per completed plotted cell for FastGraph/PCA, FAISS, cuVS BF, and GGNN; CAGRA-NN-Descent has three. Inputs are copied to a Torch CUDA tensor before timing: FastGraph and GGNN consume that tensor directly, FAISS uses its Torch bridge, and cuVS BF and CAGRA use a pointer-identical CuPy DLPack view created before the timer. The timed region therefore includes index construction and search, but excludes data loading and host-to-device transfer for every backend. All timed regions are bracketed by device synchronization. Figures and scalar comparisons report medians across successful repetitions; error bars show the interquartile range.

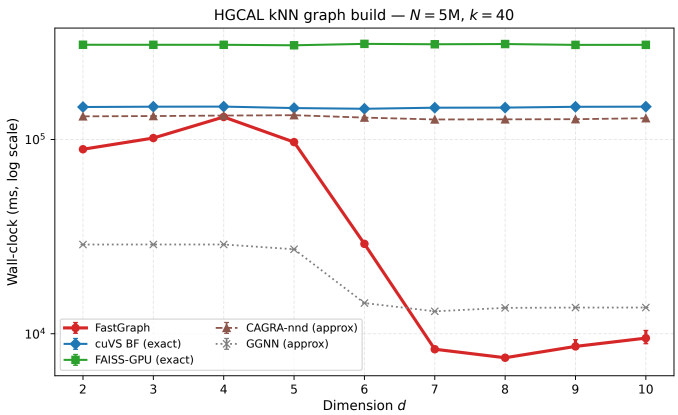
CAGRA build path. The default CAGRA IVF-PQ build path fails in the 5 M-point timing campaign at $d \geq 5$ with a RAFT assertion about invalid or duplicated neighbor nodes. We did not isolate the internal cause of this failure. We therefore use CAGRA's `build_algo="nn_descent"` path for the headline timing campaign; it avoids the PQ initialization, succeeds across the full d=2–10 sweep, and is the robust full-sweep CAGRA timing baseline. The archived recall diagnostic predates that campaign and is labeled separately as default-build CAGRA.

The `max_bin_dims` hyperparameter. The binning subspace size k is exposed as `max_bin_dims`. The benchmark binary and evaluated kernel surface provide compile-time template instantiations for $k \in \{2, 3, 4, 5\}$; we use $k = 3$ as the default for HGCAL and ablate that surface in Section 5.5. The immutable source release also contains experimental $k = 6, 7$ dispatches used to probe the design boundary, but they are not part of the recommended headline configuration; $k = 7$ exceeds the current per-block register budget for some configurations. The empirical optimum on HGCAL data is well inside the evaluated range.

Benchmark data. The detector-derived benchmark matrix comprises 1.16 billion CMS HGCAL simulation recHits concatenated across events, with ten stored feature columns per hit. The harness deterministically takes the first N rows and first d columns; the available artifact does not identify the column names, units, preprocessing, or normalization. Increasing d therefore changes both the number and identity of included features, and PCA remains sensitive to their numerical scales. Each timed slice is passed to the implementations as one segment rather than preserving event boundaries. This construction is a large-N stress test of the kernels on detector-derived features; it is not an event-batched end-to-end GNN workload or a learned-latent sample. Synthetic comparisons use isotropic Gaussian $\mathcal{N}(0, I_d)$ samples; this matches the benchmark protocol used in this paper and isolates the effect of dimensionality from the feature-scale effects present in the detector-derived matrix.

5. Performance Comparisons

5.1. Headline benchmark: Dimensional scaling on detector data



d	FAISS	cuVS	CAGRA	GGNN
2	3.45×	1.65×	1.48×	0.32×
3	3.02×	1.45×	1.30×	0.28×
4	2.35×	1.13×	1.02×	0.22×
5	3.15×	1.50×	1.38×	0.28×
6	10.71×	4.96×	4.47×	0.50×
7	37.17×	17.52×	15.24×	1.57×
8	41.23×	19.41×	16.88×	1.81×
9	35.68×	17.13×	14.79×	1.59×
10	32.36×	15.55×	13.56×	1.44×
geo.	10.4×	5.0×	4.4×	0.64×

Figure 1: Headline benchmark on detector data at N=5 M, k=40. Left: dimensional scaling of wall-clock time. FastGraph is the fastest among the tested dimension-general exact GPU baselines across the full d=2–10 range. Right: speedups of FastGraph over each backend at the same operating point. FAISS and cuVS BF are exact; CAGRA-nnd and GGNN are approximate default-recall baselines, and GGNN's timing should be read with its substantially lower recall in this regime (Section 5.7).

The right-hand table in Figure 1 reports the cross-method speedups of FastGraph at the headline operating point. FastGraph is the fastest among the tested dimension-general *exact* GPU kNN baselines at every dimension we measure and has a 4.4× geomean advantage over the approximate CAGRA-NN-Descent build. The GGNN approximate baseline does win on raw wall-clock at low d. Section 5.7 reports recall separately for the available diagnostic configurations. The relative advantage is widest in the d=7–10 regime where dense exact baselines pay the full quadratic cost and the approximate baselines have to retain enough graph quality to remain useful. Geomean across d=2–10 at this operating point is 10.4× over FAISS, 5.0× over cuVS BF (the strongest exact GPU baseline), and 4.4× over CAGRA-NN-Descent. Section 5.5 separately reports the internal axis-aligned ablation that motivated the PCA-subspace design; the headline table therefore reports only the PCA-subspace method.

The curve shows a marked change between d=5 and d=6. Below this point, all GPU methods are within a small constant factor and the binning advantage is modest; above it, FAISS/cuVS BF (which scale as $N^2 d$) both grow rapidly with d, while FastGraph retains a useful pruning structure through PCA-subspace binning. Because each successive d

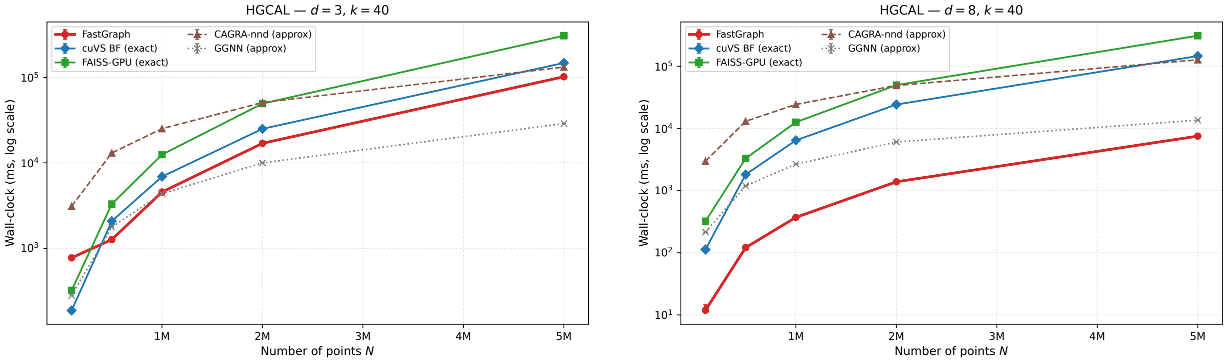


Figure 2: Dataset-size scaling on HGCAL at k=40 for d=3 (left) and d=8 (right). FastGraph maintains a speed advantage over exact baselines most clearly in the higher-dimensional PCA-favorable regime.

appends a specific raw feature column, this change cannot be attributed to dimensionality or PCA anisotropy alone without feature-subset and normalization controls.

5.2. *Dataset-size scaling*

Figure 2 reports dataset-size scaling at fixed k=40 for d=3 and d=8 on the common five-point `mps:100` grid. Separation is strongest in the PCA-favorable regime.

Figure 3 checks that the headline dimensional trend is not an artifact of the single k=40 operating point. Across $k \in \{10, 40, 100\}$, the low-dimensional constant-factor regime and the larger d≥6 speedups remain visible.

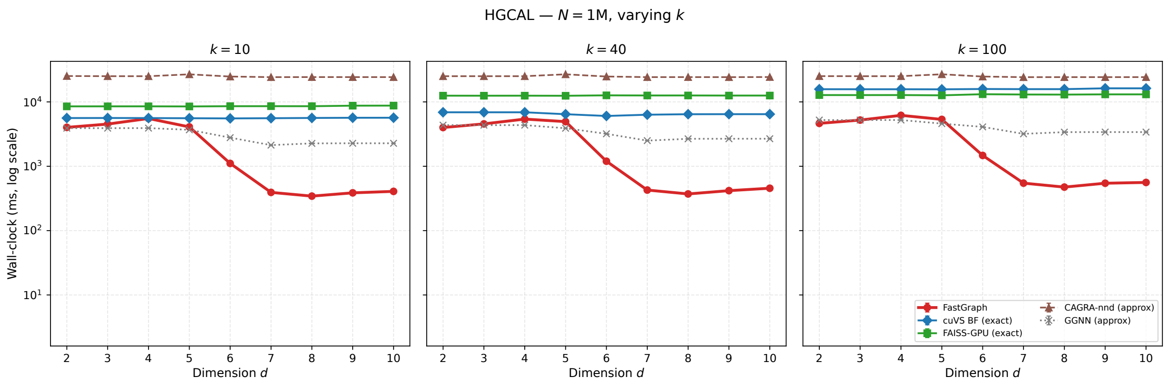


Figure 3: Wall-clock at N=1 M for $k \in \{10, 40, 100\}$ across the full dimensional range. The qualitative pattern of Figure 1 is preserved across k.

5.3. *Controlled synthetic benchmarks*

On isotropic Gaussian $\mathcal{N}(0, I_d)$ data, FastGraph wins decisively at low dimensions, while dense brute-force GPU matmul (cuVS BF) becomes competitive as d grows. This is consistent with the algorithmic interpretation: the PCA step has no anisotropy to exploit on isotropic data, so the gain comes from ordinary cell-list pruning rather than from a favorable learned basis. At low d, cell-list pruning still dominates the dense $O(N^2 d)$ matmul of the exact GPU baselines, giving FastGraph a 55× speedup over cuVS BF at d=3 and a 4.8× speedup at d=5 (Table 1). As d grows the number of neighboring cells grows combinatorially, the binning savings collapse, and dense matmul on a well-tuned BLAS path is close to optimal on isotropic data. Figure 4 shows the full dimensional sweep and two dataset-size scans at d=3 and d=8.

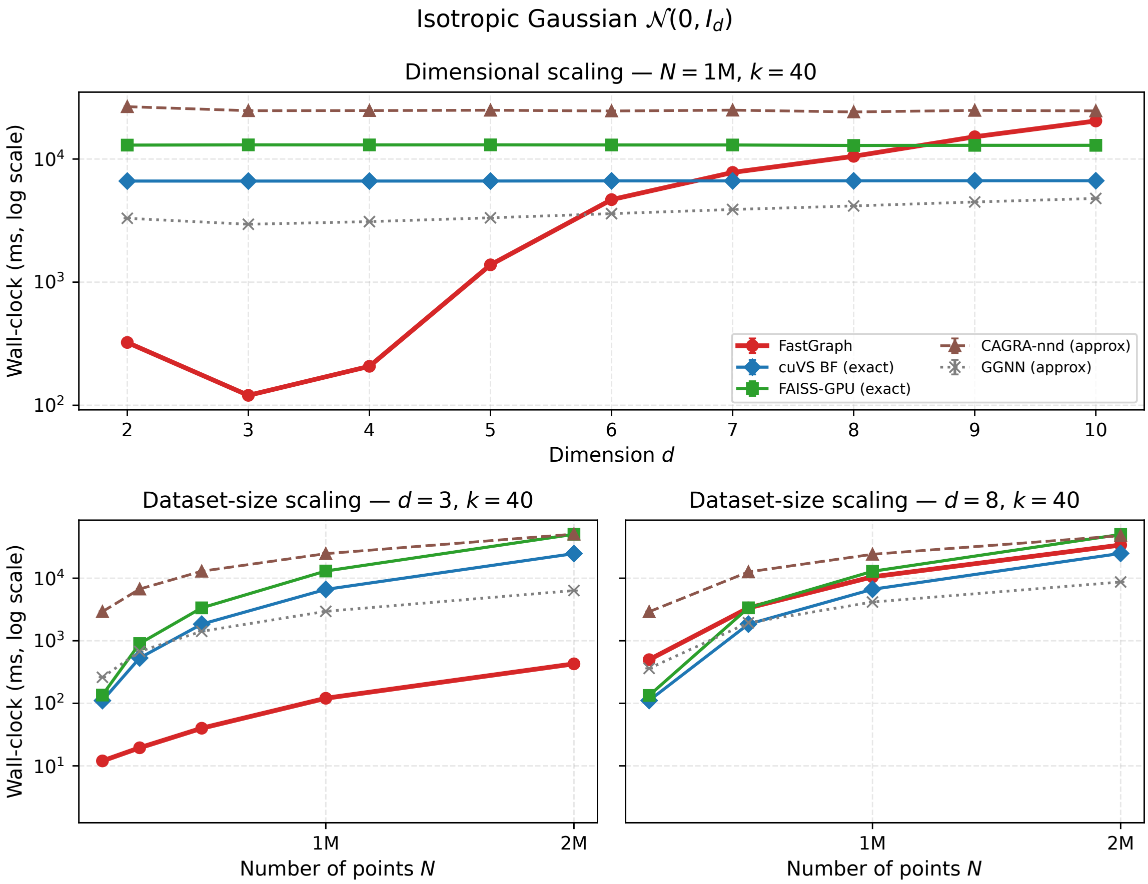


Figure 4: Controlled synthetic Gaussian benchmarks. Top: dimensional scaling at N=1 M, k=40. Bottom: dataset-size scaling at d=3 and d=8, k=40.

Table 1: FastGraph speedup over GPU baselines on synthetic Gaussian $\mathcal{N}(0, I_d)$ data at N=10^6, k=40 (median of 3 reps on the fixed-seed synthetic Gaussian benchmark set used throughout this paper). The HGCAL detector-data wins at $d \geq 6$ (Figure 1) are not reproduced on isotropic data, showing that the detector-data result does not follow from dimensionality and cell-list binning alone.

d	vs cuVS BF	vs FAISS-GPU	vs CAGRA-nnd
3	55.2×	108.6×	205.8×
5	4.8×	9.4×	18.0×
7	0.8×	1.7×	3.2×

The implication for the headline claim is narrower: the HGCAL detector wins at d=6–10 (Section 5.1) are consistent with PCA exploiting structure absent from isotropic synthetic data, but the current feature-prefix experiment does not isolate dimensionality, feature identity, units, and anisotropy from one another.

5.4. Memory footprint

Resident memory. At N=5 M, d=3, k=40, the synchronized post-call delta is about 5.15 GiB for FastGraph versus 1.58–2.82 GiB for the other backends. FastGraph returns int64 indices and float32 distances, whose N×k tensors alone occupy about 2.24 GiB (2.40 GB) at this point; the remainder includes live tensors and allocator state retained after the call. CAGRA and GGNN return different output representations, so this is an as-used resident-footprint comparison rather than an output-normalized workspace comparison.

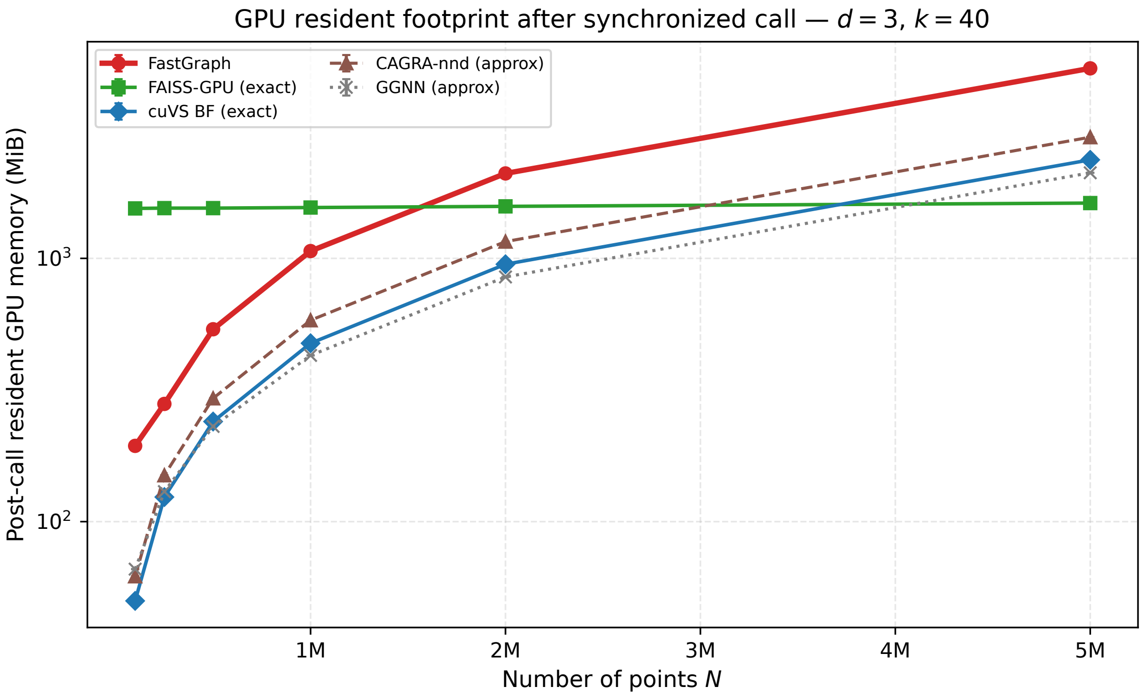


Figure 5: Post-call GPU memory at d=3, k=40. Each value is the change in device-wide NVML used memory between a pre-call baseline and a synchronized post-call sample while the caller still holds the backend's outputs. Values include live outputs plus allocator and index state; they are not transient peak-workspace measurements.

Approximate-backend interpretation. The plot reports each backend at the configuration used elsewhere in this paper. FAISS-GPU `IndexFlatL2` is exact; cuVS BF is exact. CAGRA-nn-descent and GGNN are approximate at their default quality settings (see Section 5.7). We therefore report the memory each backend uses *as benchmarked*; because the backends differ in recall and in their output and index representations, this is not an output- or quality-normalized workspace comparison.

Measurement methodology. Memory is reported from synchronized before/after NVML samples. A Python-thread poller was also recorded, but several external backends hold the Python GIL and starve that poller during the call; those transient samples are therefore excluded rather than presented as comparable peak measurements.

5.5. Binning dimensionality ablation

The binning subspace size k (`max_bin_dims`) is the most important FastGraph hyperparameter. We set the default to k=3 because it is the fastest PCA-subspace setting at the reference cell d=8, N=5 M, k_{nn}=40. Table 2 compares this choice with the axis-aligned cell list over the evaluated kernel surface $k \in \{2, 3, 4, 5\}$, using medians over three repetitions.

Both variants are non-monotonic in k, but their optima differ: the axis-aligned cell list is fastest at k=2, while the PCA-subspace variant is fastest at k=3. The largest matched-k speedup occurs at k=4, but k=3 is the setting used by FastGraph because it minimizes the PCA-subspace wall-clock. Across the sweep, PCA-subspace binning is 8.6–30.6× faster at matched k, and comparing each variant at its own optimum gives a 26.6× speedup.

We expose k as a user-facing hyperparameter rather than hard-coding it: the optimum is data-dependent (a dataset with intrinsic dimensionality near 2–3 prefers small k; one with variance spread more uniformly prefers k=4 or 5), and the same kernel binary supports all four instantiations. The default k=3 is HGCAL-tuned and should be revalidated when deploying FastGraph on datasets with different intrinsic dimension.

Table 2: Effect of the binning subspace size k on FastGraph wall-clock at d=8, N=5 M, k_{nn}=40, median of three repetitions. The axis-aligned variant's optimum within the evaluated kernel surface is k=2; the PCA-subspace variant's runtime optimum, and therefore the default used in the paper, is k=3.

k (bin dims)	axis (s)	PCA (s)	PCA/axis speedup
2	199.37	11.39	17.5×
3	204.65	**7.48**	**27.4**×
4	353.78	11.58	30.6×
5	242.90	28.10	8.6×

5.6. Three-dimensional baseline: CLOVER head-to-head

CLOVER [23] is the strongest among the 3D-only exact-GPU kNN methods surveyed in Section 2 and the only method in this group that our head-to-head shows beating FastGraph on raw 3D detector coordinates. This comparison does not weaken the headline claim: CLOVER is restricted to raw 3D point clouds, whereas FastGraph targets exact kNN in learned latent spaces with d=4–10.

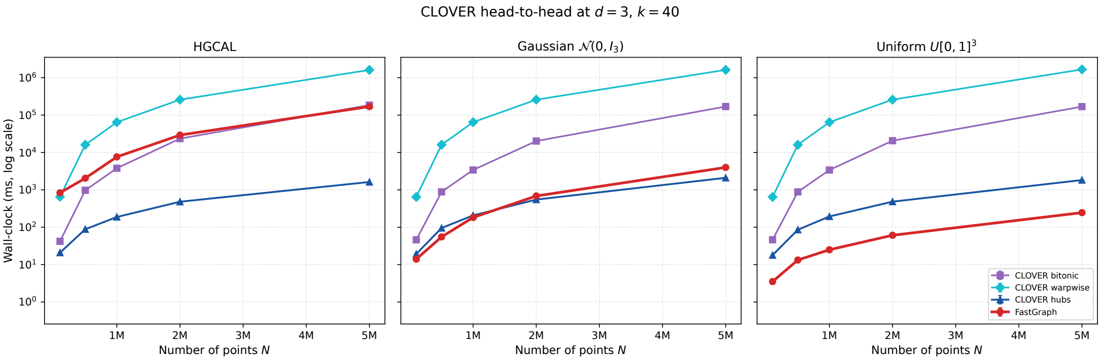


Figure 6: FastGraph vs CLOVER at d=3, k=40, across three data distributions: HGCAL recHits (left), synthetic Gaussian $\mathcal{N}(0, I_3)$ (center), synthetic uniform on $[0, 1]^3$ (right). CLOVER is only implemented for D=3, so d=3 is the only dimensionality at which a direct comparison is possible.

The result splits along the structure of the input distribution. On uniform-cube data — the regime where a uniform-grid cell list is near-optimal — FastGraph is 5–8× faster than CLOVER hubs. On isotropic Gaussian data the two methods are within roughly a factor of two. On anisotropic HGCAL recHits, CLOVER's hubs index is much faster at d=3: with the default k_{bin}=3, FastGraph short-circuits to the ordinary axis-aligned cell-list path, while CLOVER uses a 3D density-adaptive spatio-graph. This is a favorable regime for CLOVER and an intentionally out-of-scope regime for FastGraph's PCA-subspace contribution.

The deployment regime targeted by FastGraph is the *learned latent space* of a dynamic-graph network, not only raw 3D detector coordinates. The original GravNet formulation uses a four-dimensional learned space for neighbor construction [2]; this work evaluates that point and nearby low-to-moderate dimensions through d=10. CLOVER, LBVH-kNN, and the RT-core methods considered here do not provide a released $d \geq 4$ implementation for a direct comparison.

5.7. Recall evaluation

We use an archived broad recall sweep against a brute-force reference on HGCAL data up to N=5×10^5. This diagnostic invokes the axis-aligned `binned_select_knn` control rather than the PCA wrapper. Its CAGRA rows use the default graph build and `itopk_size=512`; its GGNN rows use $\tau_{query} = 0.8$. This archived experiment is separate from the headline timing campaign, which uses CAGRA–NN-Descent. We use these archived outputs only to report recall; none of their timing values enters the performance comparisons.

Because many recHits are spatially coincident, neighbor indices are not unique. We use *distance-based recall*, counting a predicted neighbor as correct when its squared distance is no larger than the K-th reference distance plus a small tolerance. FastGraph and its reference use element-wise squared ℓ_2 distances, which avoid the cancellation that can affect coincident-hit cases under the BLAS expansion. FAISS-GPU `IndexFlatL2` is exact under its own L2 kernel and is shown as such. Exactness here means satisfying the fixed distance criterion, not producing a unique graph topology when ties are present.

At the common representative cell, the FastGraph axis control attains recall = 1.0, while neither approximate backend reaches exact recall at its recorded highest-quality setting. Table 3 reports means over the three repetitions of that cell. We do not report a cross-grid mean because the archived default-build CAGRA data cover d = 2–7, whereas the GGNN and FastGraph data extend through d = 10.

Table 3: Distance-based recall at d=3, N=5 × 10^5, K=40. CAGRA uses its default graph build with `itopk_size`=512; GGNN uses τ_{query} = 0.8. Values are means over three repetitions.

Method	distance-based recall
FastGraph axis control (exact)	1.000
FAISS-GPU `IndexFlatL2` (exact)	1.000
CAGRA, default build (approximate)	0.818
GGNN (approximate)	0.410

These values establish the quality gap in the recorded configurations. None of the recorded approximate settings reaches the exact operating point.

The PCA path is validated separately with the released `binned_select_knn_pca` entry point. Dedicated GPU tests compare the complete sorted distance multiset returned for every query with an element-wise FP32 brute-force reference at $d \in \{4, 5, 8, 10\}$, $K \in \{16, 40, 128\}$, and N up to 8,000. They also exercise random PCA subsampling and a three-segment `row_splits` input. All tested indices are valid, unique, and segment-local; returned distances match recomputed distances, and every distance multiset matches brute force within the stated FP32 tolerances.

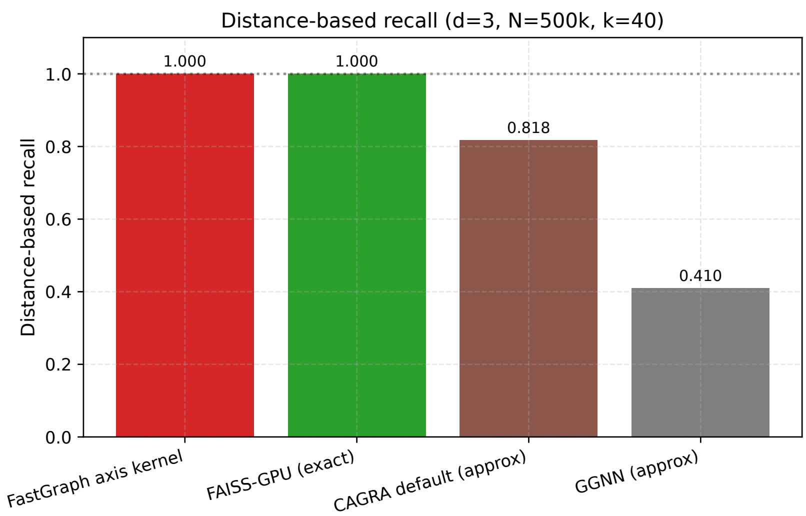


Figure 7: Distance-based recall at the representative HGCAL diagnostic cell d=3, N=5 × 10^5, K=40. FastGraph and FAISS-GPU satisfy their exact distance conventions; the approximate results use default-build CAGRA with `itopk_size`=512 and GGNN with τ_{query} = 0.8. This figure reports recall only.

5.8. Dynamic graph construction in GNNs

GravNet-style layers [2] construct a fresh kNN graph in a learned latent space at every forward pass, so the kNN substrate is on the gradient path of every training step. FastGraph integrates into this setting through `GravNetOp`, a PyTorch module that wraps coordinate transformation, kNN graph construction (via the opt-in `use_pca=True` PCA-subspace path of Section 3), and message passing into a single autograd-compatible operation. Gradients flow through the selected distances and the input coordinates; the discrete neighbor indices and the per-call projection V_k are held fixed across each forward–backward pair so that graph selection is consistent within the pair; atomic gradient accumulation is not guaranteed to be bitwise deterministic. Object condensation [3], which is the loss used together with GravNet for HGCAL reconstruction, is supported through a separate GPU-resident library helper.

6. Limitations

Data-adaptive binning subspace. Both FastGraph and its axis-aligned ablation bound the grid at n_{bins}^k for a small binning dimension k, independent of input dimension d. FastGraph uses a data-adaptive orthonormal basis rather than tying the grid to a fixed set of input columns. This removes dependence on column order for a fixed feature matrix, but PCA is not invariant to rescaling individual features: their units and preprocessing change the covariance and therefore the selected subspace. The method still requires a small k to keep the grid tractable.

Data distribution still matters. The PCA contribution's effectiveness is a function of how much pairwise-distance variance the leading components actually capture. On the anisotropic detector-feature matrix used here, FastGraph's PCA-subspace binning is substantially faster than axis-aligned cell lists (Section 5.5, geomean speedups in Section 5.1). On isotropic data, the leading subspace is an arbitrary orthonormal k-dimensional projection and FastGraph degrades gracefully toward ordinary cell-list behavior; in that regime FastGraph outperforms dense GPU brute force only while the cell-list pruning advantage outweighs the constant factor of an optimized BLAS `gemm`. Section 5.3 documents this on Gaussian $\mathcal{N}(0, I_d)$ inputs and is the reason we do not claim FastGraph is the fastest GPU exact kNN on arbitrary distributions; we claim it is the fastest among the tested exact GPU baselines on HGCAL-like workloads where the leading principal components carry real structural information. Whether trained learned-latent embeddings exhibit similarly favorable spectra remains to be tested.

Remaining limitations. PCA estimation adds a sampled low-rank fit and an $O(Ndk)$ projection that are amortized over the kNN query in the regimes we benchmark, but at very small N the projection cost becomes a non-negligible fraction of the total wall-clock; for tiny per-event point counts an axis-aligned variant remains preferable. The current implementation targets a single GPU; multi-GPU or distributed kNN graph construction is not supported. The discrete neighbor indices are not differentiable; FastGraph's autograd contract is over the continuous distances and coordinates only, matching every other hard-selection kNN layer. Approximate methods can be faster in some cells at lower recall: GGNN wins on raw wall-clock for d=2–6, whereas FastGraph is faster than GGNN for d=7–10 and faster than CAGRA throughout the headline sweep.
The appropriate backend therefore depends on dimension, required graph quality, and end-to-end cost.

The detector-derived timing study is a kernel stress test rather than a deployment benchmark: it concatenates recHits and passes each slice as a single segment. Event-preserving row splits can change both the problem size and the spatial density seen by a cell list. Likewise, the benchmark features are detector-derived inputs, not embeddings sampled from a trained model. Event-batched timings and learned-latent coordinates are therefore required before extrapolating the reported speedups to end-to-end training throughput. We defer those model-level measurements to future work and make no end-to-end throughput claim here. Extending the evaluated input dimension beyond d=10 is secondary to those two validation steps; higher-dimensional feature-space graph layers remain outside the present claim. A projected-space LBVH is another possible extension when uniform bins cease to prune effectively.

Why exactness matters for the downstream GNN. The case for an exact kNN backend over a recall-tuned approximate one is not a raw-recall argument but a sensitivity-of-downstream argument. Adversarial-robustness studies of graph neural networks show that node predictions can shift substantially under small, structure-preserving edge perturbations to the input graph [8]; the same literature also documents that systematic reweighting of the neighborhood—e.g., replacing direct neighbors with a diffusion-smoothed neighbor set—measurably alters downstream accuracy [9]. The neighbor-set mismatches injected by an approximate kNN are analogous edge perturbations and depend on build-side index hyperparameters; their event-level distribution and downstream effect have not been measured here. We do not claim the HGCAL reconstruction targets are unusually sensitive to approximate-kNN edge noise—that would require a dedicated model study beyond the scope of this paper—but the more conservative choice for a physics application that demands reproducible, systematics-controlled inference is the exact graph, particularly when an exact backend is competitive on wall-clock in the relevant operating regime (Section 5.1).

CAGRA on heterogeneous data. The default CAGRA IVF-PQ build path fails in the 5 M-point timing campaign at $d \geq 5$ (Section 4); the NN-Descent fallback succeeds across the full dimensional sweep but is 16.88× slower than FastGraph at the d=8 reference cell. We use the NN-Descent numbers for the headline timing campaign; the archived recall diagnostic is explicitly labeled as default-build CAGRA. This is an observed compatibility issue for the tested dataset and software configuration, not evidence of a generic CAGRA limitation or a diagnosis of the internal failure cause.

7. Data and Code Availability Statement

The FastGraph implementation described in this paper is available at `https://github.com/AgarwalAarush/FastGraphCompute`, as a fork of the upstream `FastGraphCompute` library. The immutable `pca-fgc-paper-v1.3.4` release tags pin the exact FastGraph, benchmark-harness, plotting, manuscript, and CLOVER-fork revisions used for this submission; the release is built against the CUDA 12.1 toolchain and reports FastGraphCompute package version `1.2.0`. The evaluated and recommended kernel surface covers binning subspace size $k \in \{2, 3, 4, 5\}$; the tag also contains experimental $k = 6, 7$ dispatches, which are excluded from the headline configuration. The benchmarking harness and runner scripts used to produce the figures live in a companion repository, `https://github.com/AgarwalAarush/fgc-performance`. The CLOVER head-to-head used a fork of `https://github.com/ampslab/clover-knn` at `https://github.com/AgarwalAarush/clover-knn` with local source modifications (`k_values` aligned to the paper's k=40, the hardcoded synthetic-N template chain in `main()` disabled in favor of the mesh path, and the build retargeted to `sm_80` for A100).

The detector-derived stress tests use CMS HGCAL simulation data (recHit feature vectors) accessed via an internal collaboration dataset of 1.16 billion calorimeter hits with up to ten stored numerical features per hit; their names and units are not included in the available artifact. The benchmark concatenates events and does not release event boundaries or learned model embeddings. The data itself is internal to the CMS collaboration; synthetic Gaussian and uniform datasets used in Sections 5.3 and 5.6 are generated on the fly from fixed random seeds and are fully reproducible from the released code. CMS collaboration members can reproduce the HGCAL results directly; external parties should contact the corresponding author.

8. Conclusion

We have presented FastGraph, an exact k-nearest-neighbor graph-construction primitive for GPU geometric deep learning. FastGraph uses a data-adaptive PCA subspace for cell-list pruning, while a contraction argument ($V_k V_k^\top \preceq I_d$) preserves exactness in the original input space. On the detector-derived recHit stress test, across d=2–10 at the headline single-segment N=5 M, k=40 operating point, FastGraph is the fastest among the tested dimension-general *exact* GPU kNN baselines at every tested dimension, with speedups up to 41× over FAISS-GPU `IndexFlatL2`, 19× over cuVS brute force
(the strongest exact GPU baseline), and 17× over the approximate CAGRA-NN-Descent build—the CAGRA path that completed the full d=2–10 sweep in our harness. The ablation in Section 5.5 isolates the PCA rotation rather than incidental implementation differences as the primary contribution: comparing each variant at its own optimum gives a within-paper ratio of 26.6× between PCA-subspace and axis-aligned binning at d=8, N=5 M, k=40.

This paper does not claim that FastGraph is the fastest GPU kNN on every distribution or every dimensionality. Two boundaries of the claim are documented. First, the method targets the d=4–10 low-to-moderate-dimensional regime around GravNet's learned space, but the current timing data are detector-derived stress-test features rather than learned embeddings; wider feature-space graph layers and trained-model coordinates require separate evaluation. The 3D-only spatial-acceleration methods (CLOVER, LBVH, RT-cores) are unavailable in the $d \geq 4$ regime by construction, and at the largest d=3 HGCAL head-to-head point (N=5 M, k=40), the CLOVER spatio-graph index is about 100× faster than FastGraph (Section 5.6). In that case FastGraph is the ordinary 3D axis-aligned cell list, not the PCA-subspace path. The d=3 detector-data case does not overlap with the regime FastGraph is designed for, but it delineates the boundary of the claim. Second, on *isotropic* data the PCA projection has no structure to exploit and FastGraph degrades toward ordinary cell-list behavior; for that case the relative ordering against dense BLAS brute force is determined by the cell-list pruning advantage alone, not by the PCA contribution. The detector-feature gains are consistent with an anisotropic-data advantage, subject to the feature-scale and feature-prefix controls identified above, and PCA-subspace binning should be a generally useful substrate for dynamic graph layers in scientific GNNs whose latent spaces have similar spectral structure.

Declaration of Generative AI and AI-Assisted Technologies in the Manuscript Preparation Process

During the preparation of this work, the authors used OpenAI Codex for language editing, software review, and reproducibility checks. The authors reviewed and edited the output as needed and take full responsibility for the content of the published article.

References

[1] R. Ying, R. He, K. Chen, P. Eksombatchai, W. L. Hamilton, J. Leskovec, Graph convolutional neural networks for web-scale recommender systems, in: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2018, pp. 974–983.

[2] S. R. Qasim, J. Kieseler, Y. Iiyama, M. Pierini, Learning representations of irregular particle-detector geometry with distance-weighted graph networks, The European Physical Journal C 79 (7) (2019) 608.

[3] J. Kieseler, Object condensation: One-stage grid-free multi-object reconstruction in physics detectors, graphs, and images, The European Physical Journal C 80 (9) (2020) 886.

[4] S. R. Qasim, K. Long, J. Kieseler, M. Pierini, R. Nawaz, Multi-particle reconstruction in the high granularity calorimeter using object condensation and graph neural networks, in: EPJ Web of Conferences, Vol. 251, 2021, p. 03072. doi:10.1051/epjconf/202125103072.

[5] S. Bhattacharya, et al., GNN-based end-to-end reconstruction in the CMS phase-2 high-granularity calorimeter, arXiv preprint arXiv:2203.01189 (2022).

[6] Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, J. M. Solomon, Dynamic graph CNN for learning on point clouds, ACM Transactions on Graphics 38 (5) (2019) 1–12.

[7] H. Qu, L. Gouskos, ParticleNet: Jet tagging via particle clouds, Physical Review D 101 (5) (2020) 056019.

[8] D. Zügner, A. Akbarnejad, S. Günnemann, Adversarial attacks on neural networks for graph data, in: Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD), ACM, 2018, pp. 2847–2856. doi:10.1145/3219819.3220078.

[9] J. Gasteiger, S. Weißenberger, S. Günnemann, Diffusion improves graph learning, in: Advances in Neural Information Processing Systems 32 (NeurIPS), 2019, pp. 13333–13345, arXiv:1911.05485.

[10] L. Verlet, Computer "experiments" on classical fluids. i. thermodynamical properties of Lennard-Jones molecules, Physical Review 159 (1) (1967) 98–103. doi:10.1103/PhysRev.159.98.

[11] M. P. Howard, J. A. Anderson, A. Nikoubashman, S. C. Glotzer, A. Z. Panagiotopoulos, Efficient neighbor list calculation for molecular simulation of colloidal systems using graphics processing units, Computer Physics Communications 203 (2016) 45–52. doi:10.1016/j.cpc.2016.02.003.

[12] J. Johnson, M. Douze, H. Jégou, Billion-scale similarity search with GPUs, IEEE Transactions on Big Data 7 (3) (2021) 535–547.

[13] J. Feydy, J. Glaunès, B. Charlier, M. Bronstein, Fast geometric learning with symbolic matrices, in: Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 14448–14462.

[14] A. Dashti, I. Komarov, R. M. D'Souza, Efficient computation of k-nearest neighbour graphs for large high-dimensional data sets on GPU clusters, PLOS ONE 8 (9) (2013) e74113.

[15] I. Komarov, A. Dashti, R. M. D'Souza, Fast k-NNG construction with GPU-based quick multi-select, PLOS ONE 9 (5) (2014) e92409.

[16] M. Fey, J. E. Lenssen, Fast graph representation learning with PyTorch Geometric, arXiv preprint arXiv:1903.02428 (2019). doi:10.48550/arXiv.1903.02428.

[17] S. R. Qasim, Multi-particle reconstruction with dynamic graph neural networks, Ph.D. thesis, Manchester Metropolitan University, see §8.2.2 "Binned K Nearest Neighbours (KNN)", Algorithm 3, pp. 154–158 (2023). URL https://e-space.mmu.ac.uk/632184/

[18] R. F. Sproull, Refinements to nearest-neighbor searching in k-dimensional trees, Algorithmica 6 (1991) 579–589. doi:10.1007/BF01759061.

[19] Y. Zhu, RTNN: Accelerating neighbor search using hardware ray tracing, in: Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP), ACM, 2022, pp. 76–89. doi:10.1145/3503221.3508409.

[20] V. Nagarajan, D. Mandarapu, M. Kulkarni, RT-kNNS unbound: Using RT cores to accelerate unrestricted neighbor search, in: Proceedings of the 37th ACM International Conference on Supercomputing (ICS), ACM, 2023, pp. 289–300. doi:10.1145/3577193.3593738.

[21] D. K. Mandarapu, V. Nagarajan, A. Pelenitsyn, M. Kulkarni, Arkade: k-nearest neighbor search with non-Euclidean distances using GPU ray tracing, in: Proceedings of the 38th ACM International Conference on Supercomputing (ICS), ACM, 2024, pp. 14–25. doi:10.1145/3650200.3656601.

[22] J. Jakob, M. Guthe, Optimizing LBVH-construction and hierarchy-traversal to accelerate kNN queries on point clouds using the GPU, Computer Graphics Forum 40 (1) (2021) 124–137.

[23] V. Kamel, H. Yan, S. Chester, CLOVER: A GPU-native, spatio-graph-based approach to exact kNN, in: Proceedings of the 39th ACM International Conference on Supercomputing (ICS), ACM, 2025, pp. 236–249. doi:10.1145/3721145.3730415.

[24] W. Zhao, S. Zhang, Y. Ding, Z. Tan, J. Liu, D. Zhan, SONG: Approximate nearest neighbor search on GPU, in: 2020 IEEE 36th International Conference on Data Engineering (ICDE), IEEE, 2020, pp. 1033–1044.

[25] F. Groh, L. Ruppert, P. Wieschollek, H. P. A. Lensch, GGNN: Graph-based GPU nearest neighbor search, IEEE Transactions on Big Data 9 (1) (2023) 267–279.

[26] H. Ootomo, A. Naruse, C. Nolet, R. Wang, T. Feher, Y. Wang, CAGRA: Highly parallel graph construction and approximate nearest neighbor search for GPUs, in: 2023 IEEE International Conference on Big Data (BigData), IEEE, 2023, pp. 33–42.

[27] H. Wang, W.-L. Zhao, X. Zeng, Large-scale approximate k-NN graph construction on GPU, arXiv preprint arXiv:2103.15386 (2021).

[28] Y. A. Malkov, D. A. Yashunin, Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs, IEEE Transactions on Pattern Analysis and Machine Intelligence 42 (4) (2020) 824–836.

[29] R. Guo, P. Sun, E. Lindgren, Q. Geng, D. Simcha, F. Chern, S. Kumar, Accelerating large-scale inference with anisotropic vector quantization, in: Proceedings of the 37th International Conference on Machine Learning (ICML), 2020, pp. 3887–3896.

[30] E. Bernhardsson, Annoy: Approximate nearest neighbors in C++/Python optimized for memory usage and loading/saving to disk, Open-source software library, Spotify, version 1.17.3 (2013). URL https://github.com/spotify/annoy

[31] X. Ju, et al., Performance of a geometric deep learning pipeline for HL-LHC particle tracking, The European Physical Journal C 81 (10) (2021) 876.

[32] A. Lazar, et al., Accelerating the exa.TrkX pipeline, Computing and Software for Big Science 6 (1) (2022) 1–9.

[33] N. Choma, et al., Track seeding and labelling with embedded-space graph neural networks, arXiv preprint arXiv:2007.00149 (2020).